LOST AND FOUND PORTAL

A Real-Time / Field-Based Research Project (22AM284) report submitted to the Jawaharlal Nehru Technological University, Hyderabad

Submitted by

RAMINENI DHARANI PRIYA 24B81A66L4
DURGA PRAVEENA SAGINENI 24B81A66L5
RAMINENI TEJASWI 24B81A66Q5

Under the guidance of
Mr.N.Nagarjuna
Sr.Assistant Professor

DEPARTMENT OF CSE (ARTIFICIAL INTELLIGENCE & MACHINE LEARNING)

CVR COLLEGE OF ENGINEERING

(***An Autonomous Institution, NAAC Accredited and Affiliated to JNTUH, Hyderabad***)
Vastunagar,
Mangalpalli(V),Ibrahimpatnam(M),
Rangareddy (D), Telangana- 501 510

APRIL 2026

CVR COLLEGE OF ENGINEERING

(***An Autonomous institution , NAAC Accredited and Affiliated to JNTUH, Hyderabad***)
Vastunagar,Mangalpalli(V),Ibrahimpatnam(M), Rangareddy
(D), Telangana- 501 510

DEPARTMENT OF CSE (ARTIFICIAL INTELLIGENCE & MACHINE LEARNING)

CERTIFICATE

This is to certify that the Real time/ Field-Based research project(22AM284) report entitled **"LOST AND FOUND PORTAL"** is a record of work carried out by **RAMINENI DHARANI PRIYA, DURGA PRAVEENA SAGINENI, RAMINENI TEJASWI** submitted to Department of **CSE (ARTIFICIAL INTELLIGENCE & MACHINE LEARNING) ,** CVR College of Engineering, affiliated to Jawaharlal Nehru Technological University, Hyderabad during the year 2024-2025.

Project Guide	**Project Coordinators**	**Head of the Department**
M.N.Nagarjuna Sr.Assistant Professor CSE(AI&ML)	**Ms.K.Anuradha** Sr.Assistant Professor **Dr.Vikram Narayandas** Associate Professor	**Dr. R. Usha Rani** Professor CSE(AI&ML)

DECLARATION

We hereby declare that the Real time/ Field-Based research project (22AM284) report entitled **"LOST AND FOUND PORTAL"** is an original work done and submitted to **CSE (ARTIFICIAL INTELLIGENCE & MACHINE LEARNING)** Department, CVR College of Engineering, affiliated to Jawaharlal Nehru Technological University, Hyderabad and it is a record of bonafide project work carried out by us under the guidance of **Mr.N.Nagarjuna**, **Sr.Assistant Professor, CSE(AI&ML).**

We further declare that the work reported in this project has not been submitted, either in part or in full, for the award of any other degree or diploma in this Institute or any other Institute or University.

Signature of the Student

RAMINENI DHARANI PRIYA

24B81A66L4

Signature of the Student

DURGA PRAVEENA SAGINENI

24B81A66L5

Signature of the Student

RAMINENI TEJASWI

24B81A66Q5

Date:

Place:

ACKNOWLEDGEMENT

We are thankful and fortunate enough to get constant encouragement, support, and guidance from all **Teaching staff of CSE (ARTIFICIAL INTELLIGENCE & MACHINE LEARNING)** Department who helped us in successfully completing this project work.

We thank **Ms.K.Anuradha and Dr.Vikram Narayandas**, Project Coordinators**,** for their valuable guidance and support which helped us to complete the project work successfully.

We respect and thank our internal guide, **Mr.N.Nagarjuna, Sr.Assistant Professor, CSE(AI&ML),** for giving us all the support and guidance, which made us complete the project duly. We would like to express heartful thanks to **Dr. R. Usha Rani**, Professor & Head of the Department, for providing us an opportunity to do this project and extending support and guidance

We would like to express heartful thanks to **Dr. H. N. Lakshmi,** Professor & Associate Dean,ET for providing us an opportunity to do this project and extending support and guidance.

We thank our Vice-Principal **Prof. L. C. Siva Reddy** for providing excellent computing facilities and a disciplined atmosphere for doing our work.

We wish a deep sense of gratitude and heartfelt thanks to **Dr. Rama Mohan Reddy**, Principal and the **Management** for providing excellent lab facilities and tools. Finally, we thank all those guidance helpful to us in this regard.

LIST OF FIGURES

Fig No.	Fig Name	Page no.
Fig 2.1	System Architecture of Lost & Found Portal	7
Fig 2.2	Data Flow Diagram	8
Fig 2.2.1	Use Case Diagram of Lost & Found Portal	9
Fig 2.2.2	Class Diagram of Lost & Found Portal	10
Fig 2.2.3	Activity Diagram of Lost & Found Portal	11
Fig 2.2.3	Sequence Diagram for Claim Process	12
Fig 3.2	Front Page Screenshots	18-20
Fig 3.3	Test Case Execution Output	21

ABSTRACT

Losing personal belongings on a college campus is a frustrating experience for students. At the same time, finding a way to return items without a reliable system can lead to confusion and misuse. Traditional methods like notice boards or word-of-mouth announcements are inefficient, lack privacy, and often fail to return items to their rightful owners. To solve this problem, we developed a Lost and Found Portal, a web-based solution designed specifically for students at our institution. The portal offers a structured digital platform with two main sections: the Lost tab and the Found tab. In the Lost tab, students who misplace an item can post details about the item type and optional descriptions. In the Found tab, students who find an item can post only the item type without giving detailed characteristics. Instead, the finder sets a minimum of five verification questions based on specific features of the item, such as brand, color, scratches, broken parts, or distinctive markings. Claimants who think the item is theirs must answer these questions, and their responses are stored safely in the database for review later.

The portal focuses on simplicity, privacy, and security. It avoids complex features like internal chat systems, image uploads, or external moderators. This makes development easier and helps prevent misuse. Without images, ownership is confirmed through specific item details rather than visual resemblance. Also, excluding chat functionality avoids unnecessary communication. Importantly, the system does not handle the verification process automatically. Instead, the finder manually reviews the claimant's answers and decides if they are correct. This allows flexibility in responses, making sure that differences in wording do not stop rightful owners from getting their items back. For instance, if the database says "it's blue in color" and the claimant answers just "blue," the finder can still accept that answer. Once the finder is happy with the claimant's responses, the system reveals the finder's phone number, allowing direct contact between the two. This setup makes the portal a trusted mediator, helping with item recovery while avoiding third parties and ensuring fairness and transparency.

The Lost and Found Portal offers a practical approach to solving a real issue in the college setting. By blending simplicity with functionality, it creates a reliable platform for students to recover their belongings securely and efficiently. The project highlights the importance of structured verification, confidentiality, and direct communication, making sure lost items are returned fairly. With its clear workflow—Lost tab for reporting, Found tab for discovery, securely stored claimant responses, and finder-led verification—the portal shows how technology can tackle everyday challenges in an effective way. This initiative demonstrates the value of digital solutions in building trust and cooperation among students, while ensuring that personal belongings are safely returned to their rightful owners.

TABLE OF CONTENTS

Chapter No.		Contents	Page No.
		Certificate	i
		Declaration	ii
		Acknowledgement	iii
		List of Figures	iv
		Abstract	v
1		**Introduction**	2
	1.1	Problem Statement	3
	1.2	Project Objectives	4
	1.3	Software &Hardware specifications	5
	1.3.1	Software requirements	5
	1.3.2	Hardware requirements	5
2		**Design Methodology**	6
	2.1	System Architecture	6
	2.2	Data flow Diagram or Flowchart	7
	2.2.1	Use Case Diagram	8
	2.2.2	Class Diagram	9
	2.2.3	Activity Diagram	10
	2.2.4	Sequence Diagram	11
	2.3	Technology Description	12-13
3		**Implementation & Testing**	14
	3.1	Code snippets	14-16
	3.2	Front Page	17-19
	3.3	Test cases	20
4		**Conclusion**	21
		Bibliography	22
		Appendix: (Source code)	23-25

1.INTRODUCTION

In today's fast-paced environments such as colleges, workplaces, and public spaces, losing personal belongings is a common issue. Items like ID cards, wallets, books, and electronic devices are often misplaced, causing inconvenience and stress to individuals. At the same time, people who find such items may not have an efficient way to return them to their rightful owners.

Traditional lost and found systems are usually informal and unorganized, relying on notice boards or word-of-mouth communication. These methods are inefficient, time-consuming, and often unsuccessful in reuniting lost items with their owners. Moreover, sharing personal contact details or images publicly can lead to privacy concerns and misuse of information.

To address these challenges, this project presents a Lost and Found Portal, a web-based application designed to provide a centralized and structured platform for reporting and retrieving lost items. The system allows users to post details about lost or found items and enables others to search and claim them through a controlled process.

A key aspect of this project is its privacy-focused approach. Unlike many existing systems, this portal avoids the use of sensitive personal data and images, thereby reducing the risk of misuse and ensuring user safety. The application is designed using modern web technologies, offering a simple, user-friendly interface along with efficient backend processing.

Overall, the proposed system aims to improve the efficiency, security, and reliability of lost and found services through a digital and privacy-conscious solution.

1.1 PROBLEM STATEMENT

In many institutions and public environments, the process of reporting and recovering lost items is largely unorganized and inefficient. Individuals who lose their belongings often have no centralized platform to report or search for them, while those who find items face difficulty in locating the rightful owners.

Existing methods, such as notice boards or informal communication, are unreliable and time-consuming. Additionally, many digital solutions require users to share personal information or images, which can lead to privacy concerns and potential misuse of sensitive data.

Therefore, there is a need for a secure, structured, and privacy-focused system that enables users to report, search, and claim lost items efficiently without exposing unnecessary personal information.

1.2 PROJECT OBJECTIVES

The main objectives of the Lost and Found Portal are:

To develop a centralized platform for reporting and managing lost and found items: This system provides a single place where users can report lost items or post found items, eliminating the need for scattered and unreliable methods like notice boards.

To enable users to easily post details of lost or found items: The application offers simple forms that allow users to enter item details such as name, description, and location in a quick and user-friendly manner.

To provide a structured mechanism for users to search and claim items: Users can browse available listings and initiate a claim process, ensuring an organized way to connect item finders with rightful owners.

To ensure user privacy by minimizing the collection and sharing of sensitive information: The system avoids the use of personal data and images, reducing the risk of misuse and protecting user identity.

To create a simple, user-friendly interface for efficient interaction: The portal is designed with a clean and intuitive layout so that users can navigate and use the system without difficulty.

To improve the chances of recovering lost items through an organized digital system: By digitizing the process, the system increases visibility and accessibility, making it easier to match lost items with their owners.

1.3 SOFTWARE & HARDWARE REQUIREMENTS

1.3.1 Software Requirements

- Operating System: Windows
  Used as the platform for developing and running the application.
- Frontend Technologies: HTML, CSS, JavaScript
  Used to design the structure, styling, and interactivity of the user interface.
- Backend Technology: Python (Flask)
  Handles server-side logic, processes user requests, and manages application flow.
- Web Browser: Google Chrome / Microsoft Edge
  Required to run and test the application efficiently.
- Code Editor: Visual Studio Code
  Used for writing, editing, and managing the project code.

1.3.2 Hardware Requirements

- Processor: Intel Core i3 or above
  Ensures smooth performance of the system.
- RAM: Minimum 4 GB
  Required for efficient multitasking and execution.
- Storage: Minimum 500 GB
  Used to store project files and necessary software.
- System Type: 64-bit Computer
  Provides better compatibility and performance.

2.DESIGN METHODOLOGY

2.1 SYSTEM ARCHITECTURE

The Lost and Found Portal follows a client-server architecture, where the user interacts with the system through a web interface, and the backend processes requests and returns responses.

The system consists of three main components:

- Frontend (Client Side): The user interface is developed using HTML, CSS, and JavaScript. It allows users to perform actions such as posting lost or found items, viewing available items, and initiating the claim process.
- Backend (Server Side): The backend is implemented using Python with the Flask framework. It handles user requests, processes data, manages application logic, and controls the flow between the frontend and the data layer.
- Data Handling: The system stores and retrieves item details such as name, description, and location. This data is used to display listings and process claims efficiently.

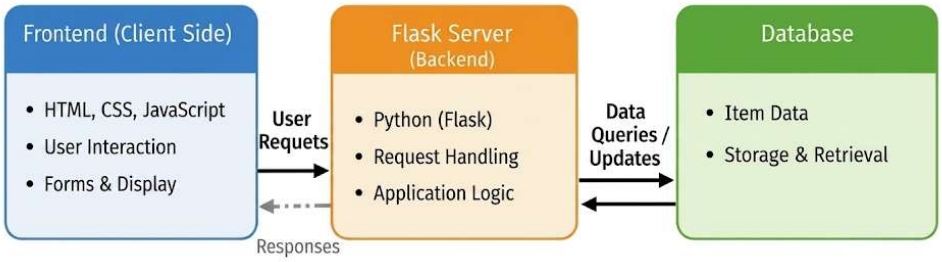


Fig 2.1: System Architecture for Lost and Found Portal

2.2 DATA FLOW DIAGRAM

The Data Flow Diagram (DFD) represents how data moves through the Lost and Found Portal system. It shows the interaction between the user, the system, and the data storage.
The main entities involved in the system are:

- User: The user interacts with the system to post lost or found items and to search or claim items.
- System (Portal): The system processes user inputs, manages item details, and controls the claim process.
- Data Storage: Stores information related to items such as name, description, and location.

The working of the system can be explained in the following steps:

- The user submits details of a lost or found item through the interface.
- The system receives the data and stores it in the database.
- The stored data is retrieved and displayed to users as item listings.
- Another user can view the items and initiate a claim request.
- The system processes the claim and updates the item status accordingly.

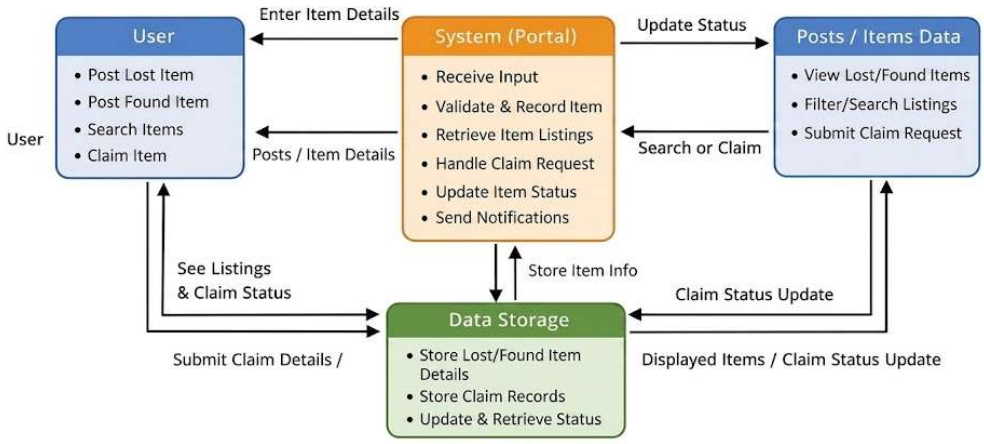


Fig 2.2: Data Flow Diagram for Lost and Found Portal

2.2.1 USE CASE DIAGRAM

A use case diagram describes how a system interacts with outside actors.

Fig 2.2.1: Use Case Diagram for Lost and Found Portal

This diagram represents a Lost and Found Portal where users can report lost items, view found items, and verify ownership through questions. It shows interactions between three roles-lost item owner, claimant, and finder, to ensure secure and accurate item recovery.

2.2.2 CLASS DIAGRAM

A class model captures the static structure of a system by characterizing the objects in the system, the relationship between the objects, and the attributes and operations for each class of objects.

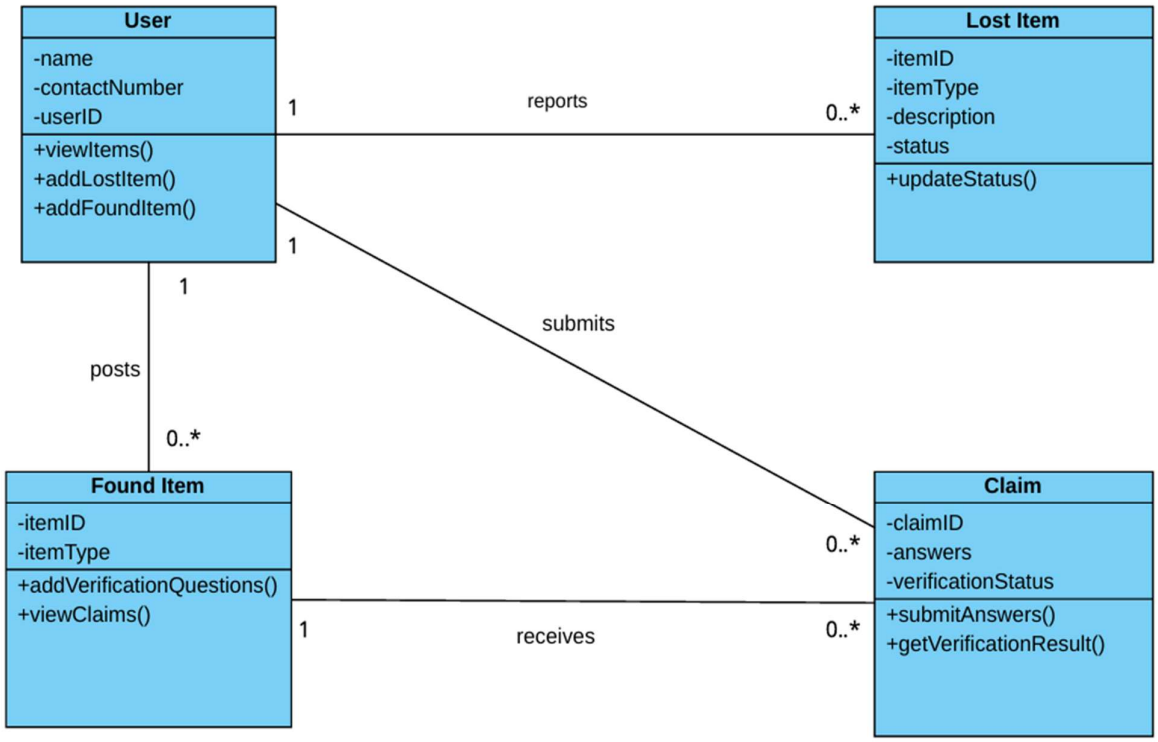


Fig 2.2.2: Class Diagram for Lost and Found Portal

This class diagram shows the structure of a Lost and Found system, including classes like User, Lost Item, Found Item, and Claim with their attributes and functions. It explains how users report or post items and submit claims, while the system manages item status and verification processes.

2.2.3 ACTIVITY DIAGRAM

An activity diagram is a type of flow chart with additional support for parallel behavior.

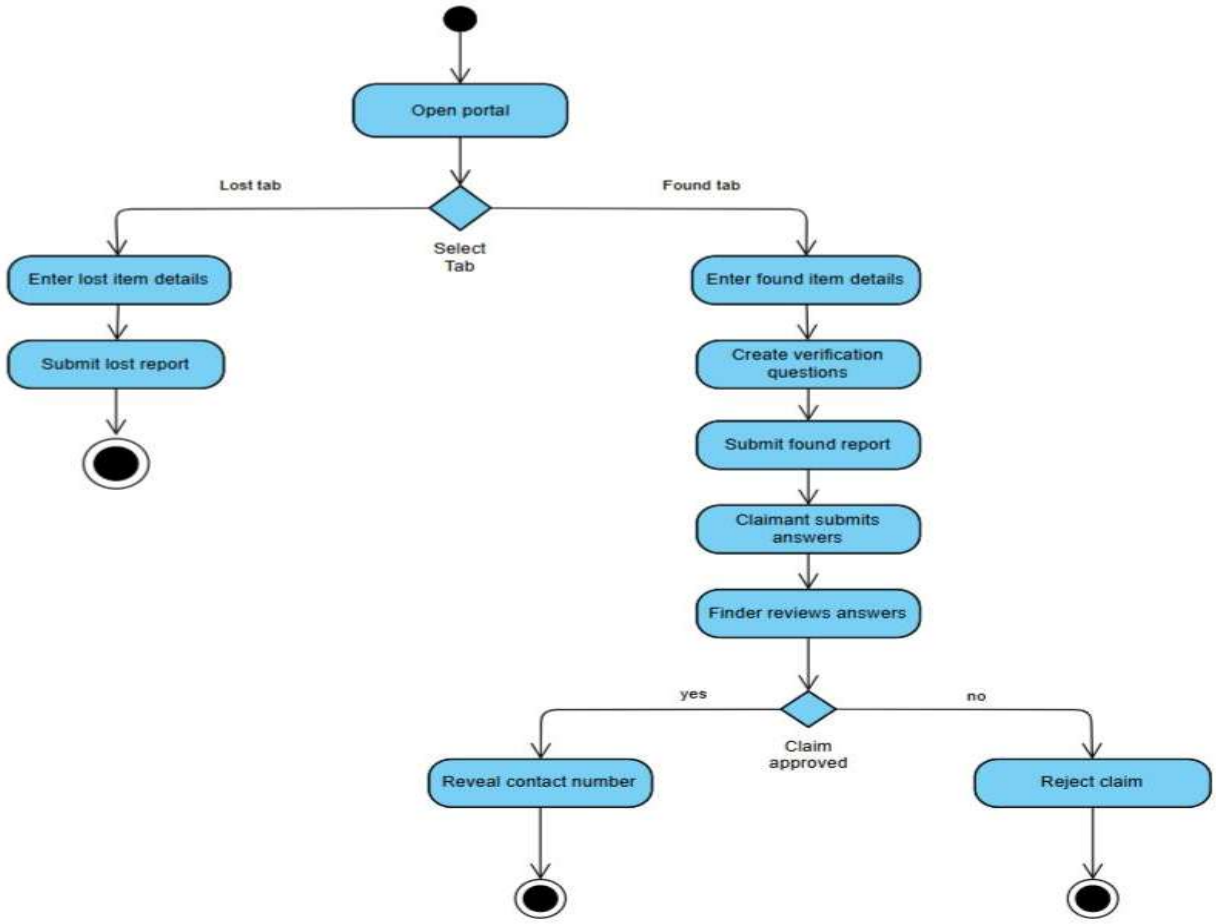


Fig 2.2.3: Activity Diagram for Lost and Found Portal

This activity diagram shows the workflow of a Lost and Found Portal, starting from opening the portal and choosing between reporting a lost item or a found item. It illustrates the process of submittingdetails, verifying claims through questions, and either approving (revealing contact) or rejecting the claim.

2.2.4 SEQUENCE DIAGRAM

The sequence diagram represents the flow of messages in the system and is also termed as an event
diagram.

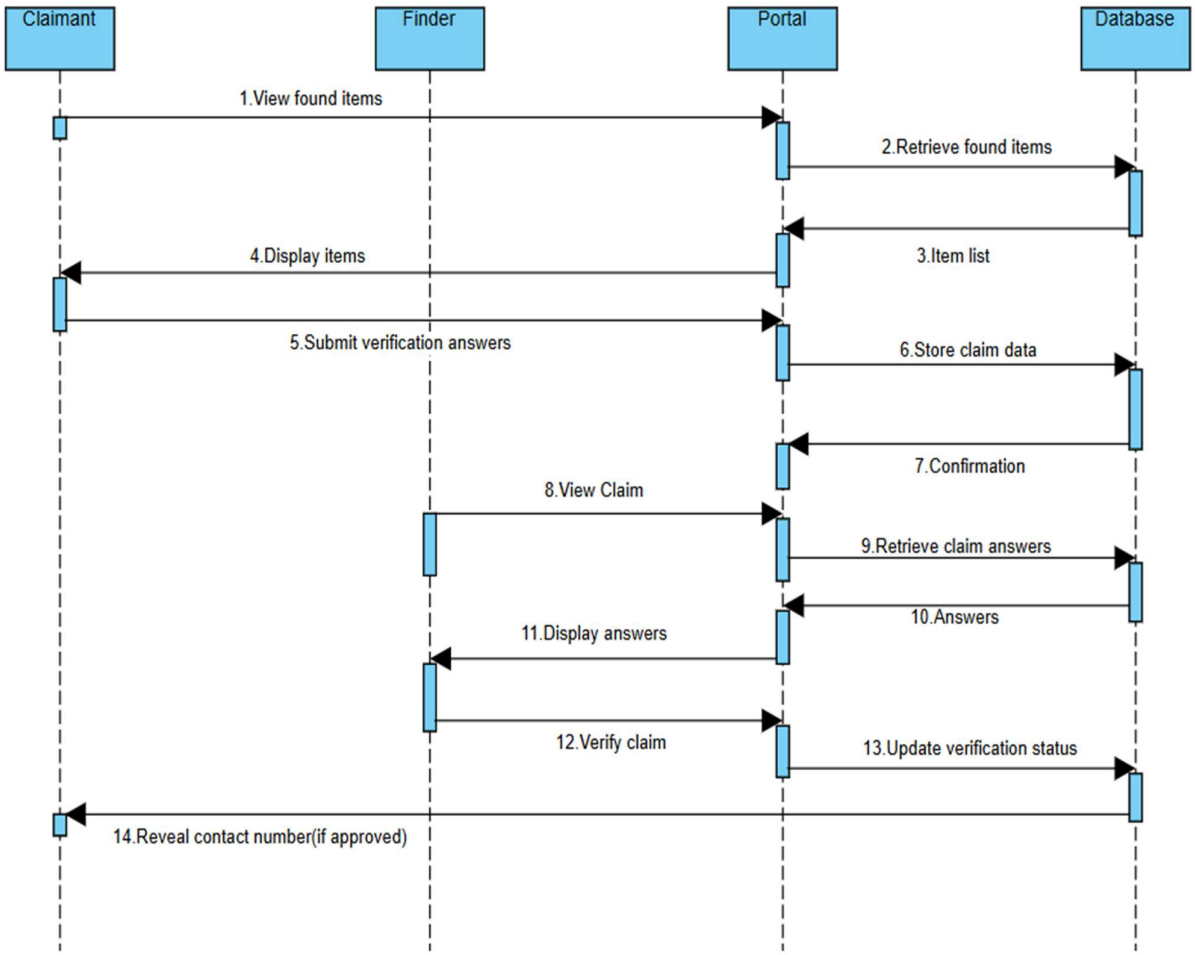


Fig 2.2.4: Sequence Diagram for Lost and Found Portal

This sequence diagram shows the interaction between claimant, finder, portal, and database while
claiming a found item. It explains how item details are retrieved, claims are submitted and verified, and the contact number is revealed if the claim is approved.

2.3 DATA DESCRIPTION

The Lost and Found Portal is developed using a combination of frontend, backend, and database technologies that work together to ensure efficient functioning of the system.

Frontend Technologies

The frontend is responsible for user interaction and presentation of the system.

- HTML (HyperText Markup Language): HTML is used to design the basic structure of the web pages. It defines elements such as forms, input fields, buttons, and headings, which allow users to enter and view information easily.
- CSS (Cascading Style Sheets): CSS is used to enhance the visual appearance of the application. It controls layout, colours, fonts, and spacing, making the interface more attractive and user-friendly.
- JavaScript: JavaScript is used to add dynamic behaviour to the web pages. It enables features such as form validation, handling button clicks, and updating content without reloading the page, thereby improving user experience.

Backend Technologies

The backend manages the core functionality and logic of the application.

- Python: Python is used as the primary programming language due to its simplicity and efficiency. It is responsible for implementing application logic, processing user inputs, and controlling the overall system behaviour.
- Flask Framework: Flask is a lightweight web framework used to build the backend of the application. It handles routing, processes HTTP requests and responses, and connects the frontend with the database. Flask helps in organizing the application structure and managing different functionalities efficiently.

Database

- PostgreSQL: PostgreSQL is used as the relational database to store and manage application data. It stores item details such as name, description, location, and status. It supports efficient data operations like inserting, retrieving, and updating records. The database is integrated with the backend to ensure reliable and persistent data storage.

Development and Deployment Tools

- Visual Studio Code: Used for writing and managing the project code.
- Git & GitHub: Used for version control and maintaining the project repository.
- Render (Deployment Platform): Used to deploy the application and host it online.
- Web Browser (Google Chrome / Microsoft Edge): Used to run and test the application.

3.IMPLEMENTATION AND TESTING

3.1 CODE SNIPPETS

1. Flask Application Initialization

```
from flask import Flask
app = Flask(__name__, static_folder=str(FRONTEND_DIR), static_url_path="")
```

Explanation:

This code initializes the Flask application. It sets up the server and links the frontend directory, allowing static files like HTML, CSS, and JavaScript to be served directly to users.

2. Database Connection Handling

```
def get_connection():
    if DATABASE_URL:
        return PostgresConnection(DATABASE_URL)

    connection = sqlite3.connect(DATABASE_PATH, timeout=5)
    connection.row_factory = sqlite3.Row
    return connection
```

Explanation:

This function establishes a connection to the database. It supports both PostgreSQL (for deployment) and SQLite (for local use), making the system flexible and adaptable across environments.

3. API to Add Lost Item

```
@app.post("/api/lost-items")
def create_lost_item():
    data = parse_json()

    with closing(get_connection()) as connection:
        cursor = connection.execute(
            """
```

```
        INSERT INTO lost_items (

  item_name, category, description, location, owner_name, contact_info, status, created_at
        )
        VALUES (?, ?, ?, ?, ?, ?, 'Open', ?)
        """,
        (
            data["item_name"],
            data["category"],
            data["description"],
            data["location"],
            data["owner_name"],
            data["contact_info"],
            utc_now(),
        ),
    )
    connection.commit()

    return jsonify({"message": "Item added successfully"})
```

Explanation:

This API allows users to report a lost item. It collects input data, stores it in the database, and confirms successful insertion. It is a core feature of the system.

4. Claim Submission Logic

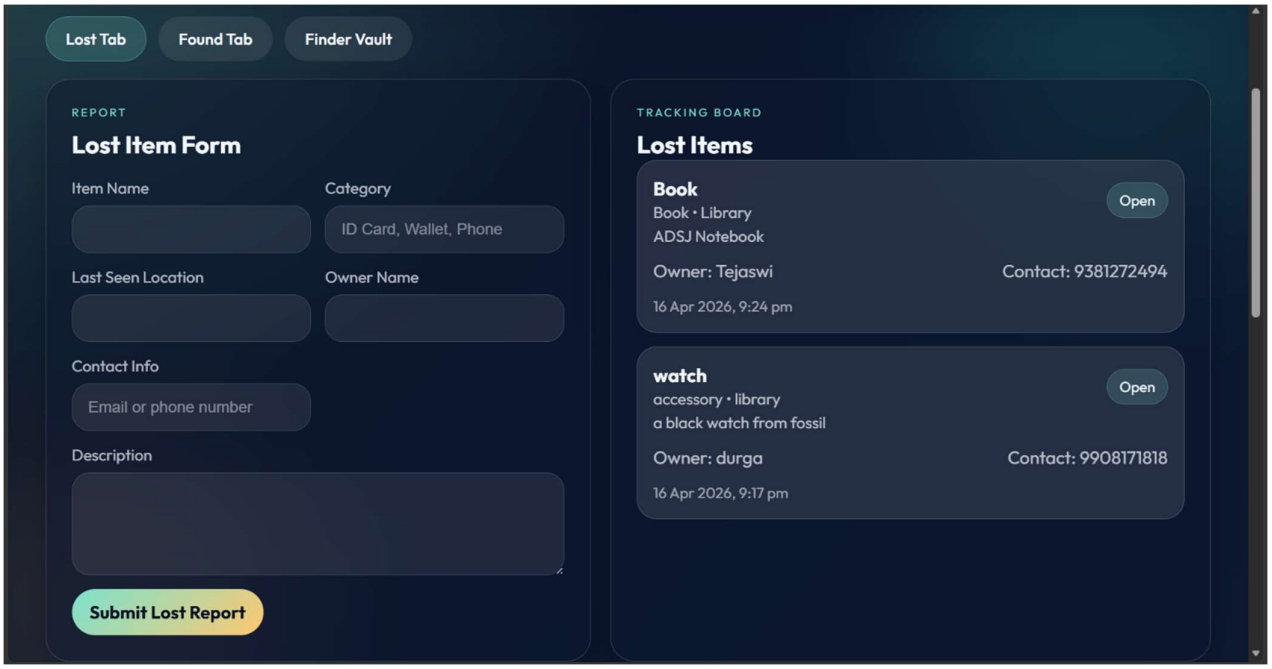
```
@app.post("/api/found-items/<int:found_item_id>/claims")
def submit_claim(found_item_id: int):
  data = parse_json()

  with closing(get_connection()) as connection:
    cursor = connection.execute(
        """
        INSERT INTO claims (
            found_item_id, claimant_name, claimant_contact, claimant_message, status, cre-
ated_at
```

```
        )
        VALUES (?, ?, ?, ?, 'Pending Review', ?)
        """,

    (
            found_item_id,
            data["claimant_name"],
            data["claimant_contact"],
            data["claimant_message"],
            utc_now(),
        ),
    )
    connection.commit()

    return jsonify({"message": "Claim submitted successfully"})
```

Explanation:

This code handles the claim process. When a user claims a found item, their details are stored in the database with a “Pending Review” status, allowing the finder to verify the request.

3.2 FRONT PAGE



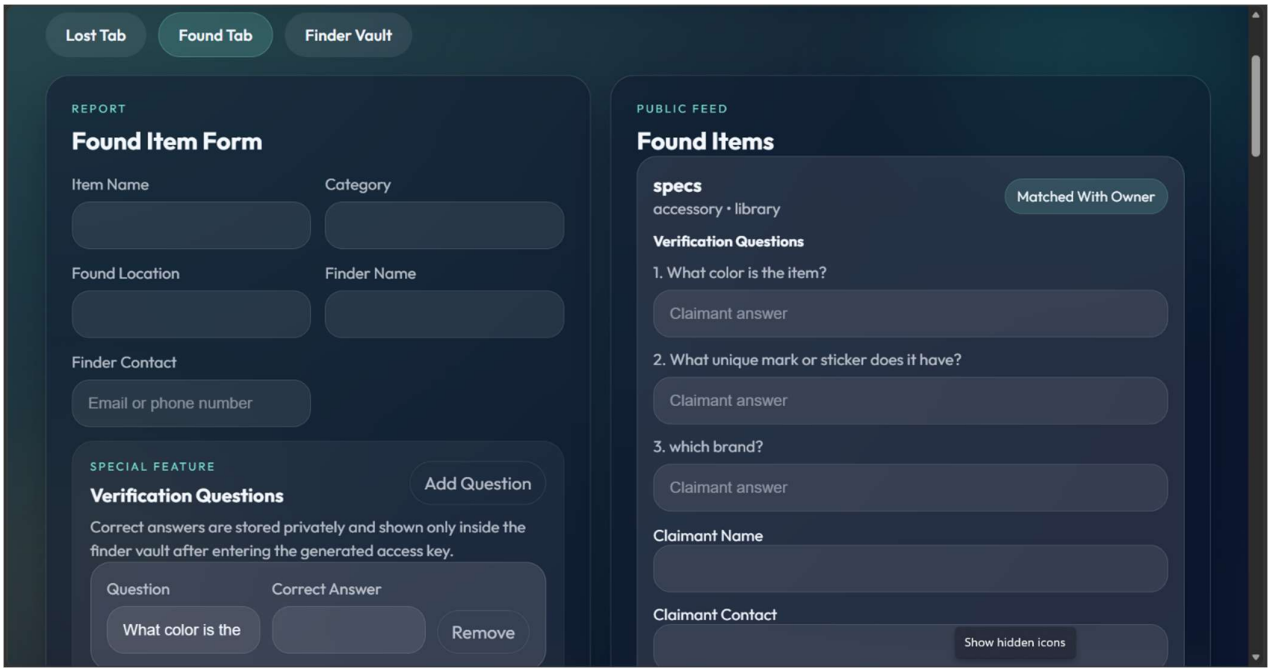


Fig 3.2: Frontpage screenshots of Lost and Found Portal

The above pictures show the Lost tab and the Found tab of the Lost and Found Portal

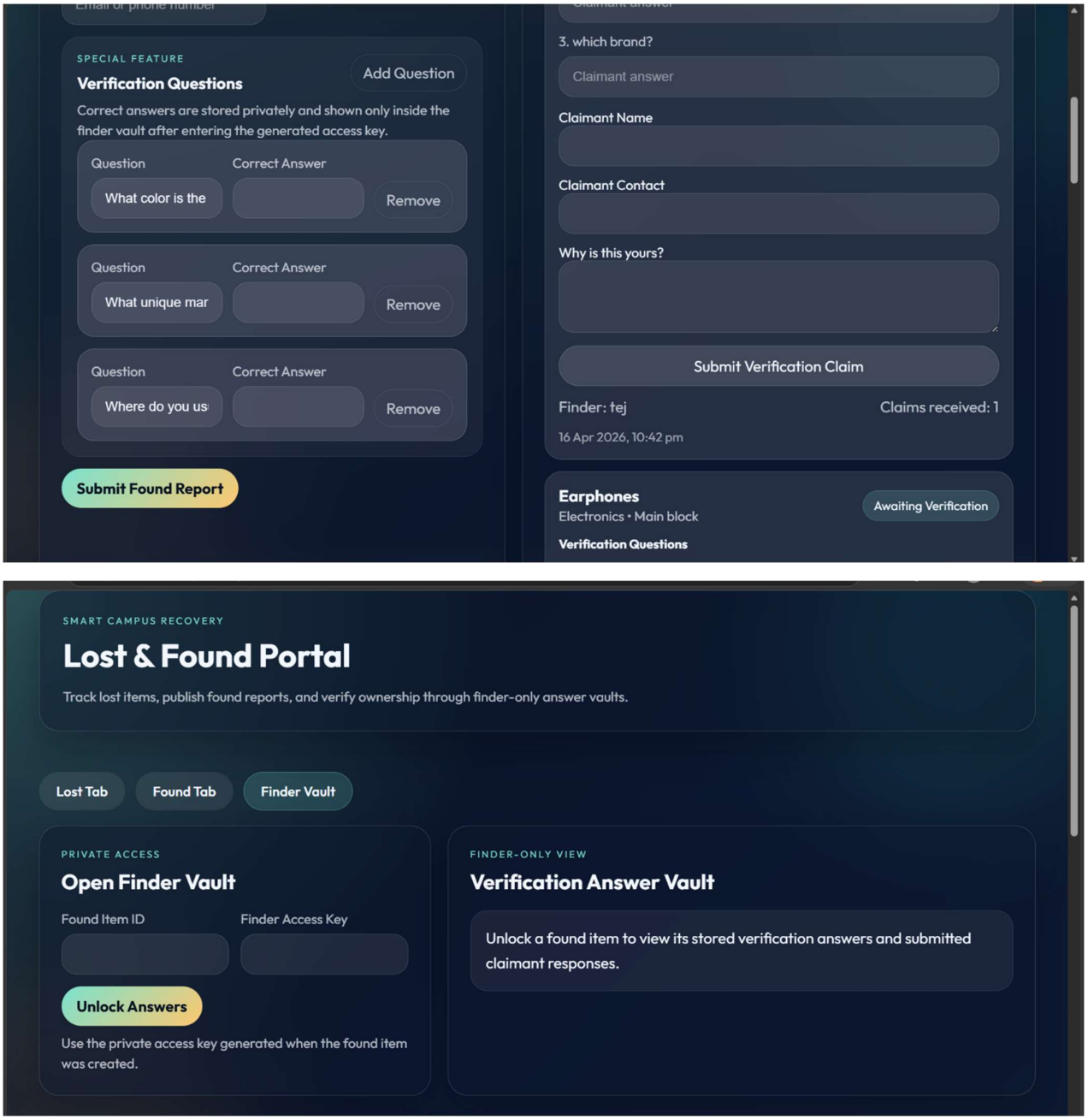


Fig 3.2: Frontpage screenshots of Lost and Found Portal

The above pictures show the verification questions feature in the Found tab and the status approval method by the finder in the Lost and Found Portal

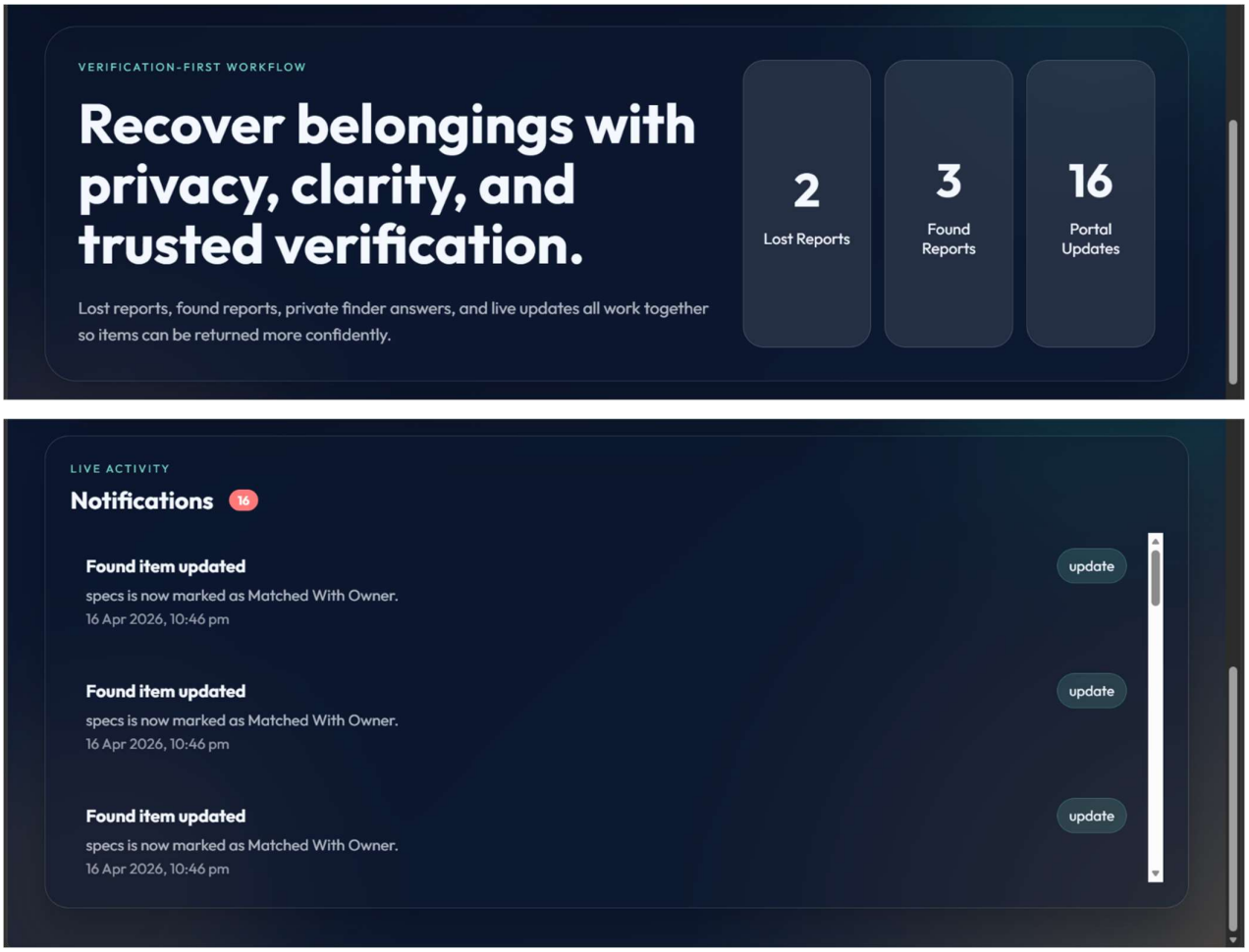


Fig 3.2: Frontpage screenshots of Lost and Found Portal

The above pictures show latest updates of the Lost and Found Portal

3.3 TESTCASES

Lost & Found Portal - Test Cases

Item Management | Claim System | Verification

Phase 1 – Item Posting & Listing

TC#	Test Case	Input	Expected Output	Status
TC-01	Add Lost Item	Valid details	Item stored in DB	Pass
TC-02	Add Found Item	Valid + questions	Item saved	Pass
TC-03	Missing Fields	Incomplete form	Error shown	Pass
TC-04	View Items	Open page	Items displayed	Pass

Lost & Found Portal - Test Cases

Item Management | Claim System | Verification

Phase 2 – Claim System

TC#	Test Case	Input	Expected Output	Status
TC-05	Submit Claim	Valid details	Stored as Pending	Pass
TC-06	Invalid Claim	Missing answers	Error shown	Pass
TC-07	Multiple Claims	Multiple users	All saved	Pass

Lost & Found Portal - Test Cases

Item Management | Claim System | Verification

Phase 3 – Verification & Status

TC#	Test Case	Input	Expected Output	Status
TC-08	Accept Claim	Valid decision	Status updated	Pass
TC-09	Reject Claim	Reject	Marked rejected	Pass
TC-10	Check Status	ID + contact	Status returned	Pass
TC-11	DB Update	After decision	Data updated	Pass

Fig 3.3: Testcase Execution output of Lost and Found Portal

4.CONCLUSION

The Lost and Found Portal developed in this project provides a structured and efficient solution for managing lost and found items in a centralized manner. The system enables users to report lost items, post found items, and claim items through a well-defined verification process.

The application successfully integrates frontend technologies such as HTML, CSS, and JavaScript with a backend built using Python and Flask. The use of a relational database ensures reliable storage and retrieval of item information, making the system consistent and efficient.

One of the key features of the system is the claim verification mechanism, which allows finders to validate claims using predefined questions. This enhances the reliability of the system and reduces the chances of incorrect claims.

The project also emphasizes simplicity and privacy, avoiding unnecessary exposure of sensitive user data while still maintaining effective communication between users. All core functionalities such as item posting, claim submission, and status updates have been tested and verified successfully.

In conclusion, the Lost and Found Portal demonstrates a practical application of web development concepts and provides a useful solution for real-world problems. The system can be further enhanced by adding features such as user authentication, notifications, and mobile support in future developments.

BIBLIOGRAPHY

The following resources were referred to during the development of the Lost and Found Portal:

1. Flask Documentation – https://flask.palletsprojects.com/
   Used for understanding Flask framework, routing, and backend development.
2. Python Documentation – https://docs.python.org/3/
   Used for learning Python programming concepts and syntax.
3. PostgreSQL Documentation – https://www.postgresql.org/docs/
   Used for database design, queries, and data management.
4. MDN Web Docs – https://developer.mozilla.org/
   Used for HTML, CSS, and JavaScript references.
5. W3Schools – https://www.w3schools.com/
   Used for basic concepts and examples of web development technologies.
6. Stack Overflow – https://stackoverflow.com/
   Used for resolving errors and debugging during development.
7. GitHub – https://github.com/
   Used for version control and project hosting.
8. Research Papers and Articles on similar websites.

APPENDIX (SOURCE CODE)

Appendix A: Project Structure

```
LOST-FOUNDPORTAL/
├── backend/
│   ├── app.py              # Main Flask application (routes + logic + database)
│   ├── portal.db           # Local database (SQLite fallback)
│
├── frontend/
│   ├── index.html          # Main UI page
│   ├── styles.css          # Styling
│   ├── script.js           # Frontend logic
│
├── requirements.txt        # Python dependencies
├── .env                    # Environment variables
├── render.yaml             # Deployment configuration
```

Appendix B: Sample Source Code

1. Backend (Flask API Snippet)

```
@app.post("/api/lost-items")
def create_lost_item():
    data = request.get_json()

    with closing(get_connection()) as connection:
        connection.execute(
            """
            INSERT INTO lost_items (
                item_name, category, description, location, owner_name, contact_info, status, cre-
ated_at
            )
            VALUES (?, ?, ?, ?, ?, ?, 'Open', ?)
            """,
            (
```

```
            data["item_name"],
            data["category"],
            data["description"],
            data["location"],
            data["owner_name"],
            data["contact_info"],
            utc_now(),
        ),
    )
    connection.commit()
```

2. Frontend (HTML Snippet)

```
<form id="lostForm">
  <input type="text" placeholder="Item Name" required>
  <input type="text" placeholder="Category" required>
  <input type="text" placeholder="Location" required>
  <button type="submit">Submit</button>
</form>
```

Appendix C: Module Description

1. Backend (Flask Application)

The backend is implemented using Flask in a single file (app.py). It handles:

- API routes
- database operations
- claim processing
- item management

2. Frontend

The frontend is developed using HTML, CSS, and JavaScript. It provides:

- item posting forms
- item listing
- claim interface

3. Database

The system uses:

- PostgreSQL (for deployment)
- SQLite (for local fallback)

It stores:

- lost items
- found items
- claims
- notifications

Appendix D: Execution Steps

```
# Install dependencies
pip install -r requirements.txt

# Run application
python app.py
```